FASE 1 · PROYECTO 03 · PROYECTO ESTRELLA

psh

Una shell POSIX con job control,
escrita desde cero en C

Versión 1.0 · Roadmap Sistemas Linux

Resumen

`psh` es la culminación de la Fase 1: una shell interactiva capaz de ejecutar pipelines, redirecciones, here-documents, variables y builtins, con manejo de señales correcto y job control completo (`&`, Ctrl+Z, `jobs`, `fg`, `bg`). Es tu primer proyecto de portfolio serio: demuestra que entiendes cómo un programa negocia con el sistema operativo procesos, terminales y señales.

CAPÍTULO I

Introducción

Una shell es, en esencia, un bucle: leer una línea, analizarla, ejecutarla y esperar. La dificultad está en los detalles. ¿Quién recibe el Ctrl+C: la shell o el programa? ¿Qué proceso controla la terminal en cada momento? ¿Cuándo hay que cerrar cada extremo de un pipe? ¿Qué pasa si un builtin aparece en mitad de un pipeline?

Usa `bash --posix` como referencia de comportamiento. Ante la duda, lo que haga bash es lo correcto, salvo que este subject diga otra cosa.

CAPÍTULO II

Instrucciones generales

- C, `cc -Wall -Wextra -Werror -std=c17`. Makefile con `all`, `clean`, `fclean`, `re`, `asan` y `test`.
- Permitido: toda la API POSIX de procesos, ficheros, señales y terminales, `readline` y `add_history` (GNU Readline), y la libc estándar. Prohibido: `system`, `popen`, `exec*p*`, `wordexp`, `glob`.
- Una sola variable global como máximo, y solo para comunicar la recepción de una señal.
- Sin fugas de memoria (`valgrind`, con supresiones solo para las fugas internas de readline) ni de file descriptors. El build `make asan` no debe mostrar ningún error.
- Arquitectura obligatoria en tres etapas separadas: **lexer → parser (AST) → ejecutor**. Nada de ejecutar mientras parseas.

> **NOTA**
> La separación lexer/parser/ejecutor no es burocracia: es lo que te permitirá añadir `&&`, `||` y subshells en el bonus sin reescribir nada, y es lo primero que mirará un revisor técnico de tu repositorio.

CAPÍTULO III

Parte obligatoria I · Ejecución

1. Bucle interactivo y modo no interactivo

- Si stdin es una terminal (`isatty`), muestra un prompt, guarda el historial y activa el job control.
- Si no lo es (`echo 'ls' | ./psh` o `./psh script.sh`), no muestres prompt ni actives job control: ejecuta línea a línea y termina con el código del último comando.
- Ctrl+D en una línea vacía sale de la shell. Si hay jobs detenidos, avisa la primera vez, como bash.

2. Lexer y gramática

Tu lexer debe reconocer palabras, operadores (`|`, `<`, `>`, `>>`, `<<`, `&`, `;`) y comillas:

- `'simples'`: todo literal, sin expansión.
- `"dobles"`: literal salvo `$`, que expande variables.
- Comillas sin cerrar: error de sintaxis (no hace falta pedir más líneas).
- Las comillas pueden concatenarse dentro de una misma palabra: `echo "a"'b'c` imprime `abc`.

Gramática mínima que debes soportar (simplificada respecto a POSIX):

```
lista       : pipeline ( (';' | '&') pipeline )* [ '&' | ';' ]
pipeline    : comando ( '|' comando )*
comando     : ( palabra | redireccion )+
redireccion : ( '<' | '>' | '>>' | '2>' | '<<' ) palabra
```

Ante un error de sintaxis, muestra `psh: syntax error near unexpected token 'X'`, no ejecutes nada de esa línea y deja `$?` a 2.

3. Expansiones

- `$NOMBRE` y `${NOMBRE}` expanden variables de entorno; `$?` expande el código de salida del último pipeline en foreground; `$$` expande el PID de la shell.
- Las variables sin comillas sufren *word splitting* por espacios: con `A="x y"`, `echo $A` recibe dos argumentos y `echo "$A"` recibe uno.
- La eliminación de comillas ocurre **después** de expandir.

4. Ejecución de comandos

- Resolución del `PATH` como en `procs` (126, 127, 128 + señal).
- Pipelines de longitud arbitraria. Todos los procesos se lanzan antes de esperar. El estado del pipeline es el del último comando.
- Redirecciones `<`, `>`, `>>`, `2>` y `<<` (here-document con delimitador; si el delimitador va entre comillas, el contenido no se expande). Se aplican de izquierda a derecha: `cmd > a > b` crea ambos ficheros y escribe en `b`.
- Los here-documents se leen **antes** de lanzar ningún proceso de la línea.

5. Builtins

Builtin	Comportamiento
`echo [-n]`	Imprime sus argumentos separados por espacios.
`cd [ruta]`	Sin argumentos va a `$HOME`; `cd -` vuelve a `$OLDPWD`. Actualiza `PWD` y `OLDPWD`.
`pwd`	Directorio actual.
`export [N=V ...]`	Sin argumentos lista las variables exportadas. Valida los nombres.
`unset N ...`	Elimina variables.
`env`	Muestra el entorno.
`exit [n]`	Sale con código `n` (módulo 256). Con argumento no numérico: error y código 2.
`jobs`, `fg`, `bg`	Ver Parte II.

> **ATENCIÓN**
> Un builtin que modifica el estado de la shell (`cd`, `export`, `exit`) solo tiene efecto si se ejecuta **en el proceso de la shell**. Dentro de un pipeline (`cd /tmp | cat`) se ejecuta en un hijo y no cambia nada, igual que en bash. Tu ejecutor debe distinguir ambos casos.

6. Señales

- En modo interactivo, la shell **nunca** muere por Ctrl+C, Ctrl+\ o Ctrl+Z: los ignora.
- Ctrl+C con el prompt vacío: nueva línea y prompt limpio, `$?` a 130.
- Ctrl+C con un programa en foreground: el programa recibe `SIGINT` (porque es el foreground process group de la terminal, no porque la shell se lo reenvíe) y la shell muestra el prompt cuando termina.
- Los hijos restauran la disposición por defecto de todas las señales (`SIG_DFL`) y desbloquean la máscara antes de `execve`.
- Si un hijo muere por `SIGQUIT` o `SIGSEGV`, muestra `Quit (core dumped)` o `Segmentation fault`, como bash.

CAPÍTULO IV

Parte obligatoria II · Job control

Aquí es donde `psh` deja de ser un ejercicio y se convierte en una shell. Lee primero el capítulo «Implementing a Job Control Shell» del manual de GNU libc: describe el diseño canónico.

- Cada pipeline es un **job** que vive en su propio grupo de procesos. El PGID es el PID del primer proceso del pipeline.
- La shell es líder de su propio grupo y toma la terminal al arrancar. Si arranca en background, debe esperar (`SIGTTIN`) hasta estar en foreground.
- Un job en foreground recibe la terminal con `tcsetpgrp`. Cuando termina o se detiene, la shell la recupera y **restaura los atributos de la terminal** (`tcgetattr`/`tcsetattr`): si `vim` se detiene, tu prompt no puede quedar en modo raw.
- `cmd &` lanza el job en background e imprime `[n] PGID`.
- Ctrl+Z detiene el job en foreground: `[n]+ Stopped cmd`.
- `jobs` lista los jobs con su número, estado (Running, Stopped, Done) y línea de comandos.
- `fg [%n]` lleva el job al foreground (enviando `SIGCONT` al grupo si estaba detenido). `bg [%n]` reanuda un job detenido en background. Sin argumento actúan sobre el job actual (`+`).
- Las terminaciones de jobs en background se notifican antes del siguiente prompt (`[n]+ Done cmd`), nunca en mitad de la salida de otro programa.
- El seguimiento de estados se hace con `waitpid(-1, ..., WNOHANG | WUNTRACED | WCONTINUED)`.

```
psh$ sleep 100 &
[1] 4211
psh$ vim notas.txt
                        # Ctrl+Z dentro de vim
[2]+  Stopped                 vim notas.txt
psh$ jobs
[1]-  Running                 sleep 100 &
[2]+  Stopped                 vim notas.txt
psh$ fg %2              # vim vuelve con la pantalla intacta
psh$ kill %1            # bonus: kill con notación %n
[1]-  Terminated              sleep 100
```

CAPÍTULO V

Tests y calidad

- Directorio `tests/` con un tester automático: para cada caso, ejecuta la línea en `psh` y en `bash --posix` en modo no interactivo y compara stdout, stderr (normalizando el nombre de la shell) y el código de salida.
- Al menos 150 casos que cubran: comillas, expansiones, redirecciones, here-docs, pipelines, errores de sintaxis, builtins y códigos de salida.
- El job control no se puede probar fácilmente sin una terminal: documenta en `tests/MANUAL.md` un guion de pruebas manuales. Bonus: automatízalo con un pseudoterminal (`openpty`/`forkpty`, o `expect`).
- `make test` ejecuta todo y falla si un solo caso falla. Configura GitHub Actions para lanzarlo en cada push, también con el build `asan`.

CAPÍTULO VI

Parte bonus

- Operadores `&&` y `||` con la precedencia correcta, y subshells con paréntesis `( ... )`.
- Expansión de comodines `*` y `?` en el directorio actual (con tu propio `opendir`/`readdir`).
- `kill %n` y `wait` como builtins.
- **Tu propio editor de línea** sin readline: terminal en modo raw con `termios`, edición con flechas, historial y Ctrl+A/Ctrl+E/Ctrl+W/Ctrl+L. Esto sí que sorprende en una entrevista.
- Un fichero `~/.pshrc` que se ejecuta al arrancar en modo interactivo.

CAPÍTULO VII

Evaluación

Prueba	Esperado
`ls -l | grep txt | wc -l`	Igual que bash.
`cat | cat | ls`	Muestra `ls` y espera; dos Enter (o Ctrl+D) para terminar, igual que bash.
`yes | head -n 3`	Termina al instante.
`echo "$HOME"'$HOME'$NOEXISTE.x`	`/home/bs$HOME.x`
`export A="a b"; ./args $A "$A"`	Tres argumentos: `a`, `b`, `a b`.
`cat << EOF` con `$USER` dentro / con `<< 'EOF'`	Expandido / literal.
`< noexiste cat | echo hola`	Error de `noexiste` y `hola`; `$?` = 0.
`ls | | wc`	Error de sintaxis; `$?` = 2; nada ejecutado.
`sleep 30`, luego Ctrl+C	Solo muere `sleep`; `$?` = 130.
`cat` + Ctrl+\	`Quit (core dumped)`, `$?` = 131; la shell sigue viva.
`sleep 50 &`, `vim`, Ctrl+Z, `jobs`, `fg %2`, `:q`	Comportamiento descrito en la Parte II; terminal intacta.
`./psh` lanzado en background desde bash (`./psh &`)	No roba la terminal ni se queda en un bucle de errores.
`echo 'echo hola; exit 3' | ./psh; echo $?`	`hola` y `3`.
`ls -l /proc/self/fd` dentro de `psh`	Solo 0, 1, 2 y el fd del directorio.
`make asan` + sesión de 5 minutos variada	Ni un solo informe de ASan/UBSan.
Preguntas: ¿por qué el hijo y el padre llaman ambos a `setpgid`? ¿Por qué la shell ignora `SIGTTOU`?	Respuesta precisa del evaluado.